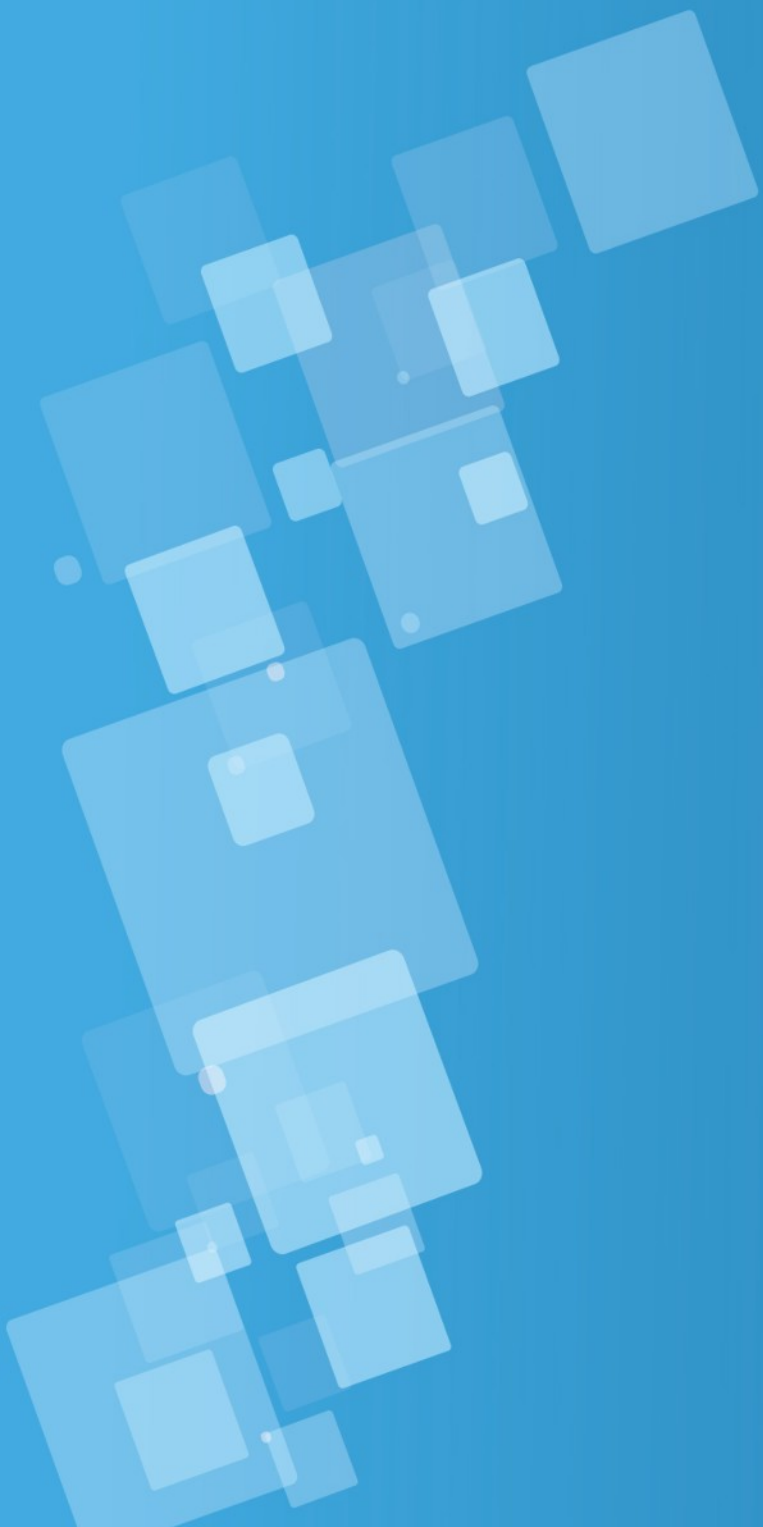




Técnicas Digitales II

RTOS (Real Time Operating System)
Sistema operativo en tiempo real
Colas (Queues)



Mecanismos de Comunicación

- Es muy común necesitar comunicar datos entre tareas.
- El método tradicional de usar variables globales, presentan varios inconvenientes:
 - Fallas por condiciones de carrera (race conditions).
 - Falta de Sincronización y Eficiencia (Polling).
 - Ausencia de Búfer y Pérdida de Datos.
 - Acoplamiento Fuerte entre Tareas.
 - Complejidad para proteger accesos concurrente.

FreeRTOS proporciona objetos de comunicación para evitar esto, el mecanismo principal y más versátil es la **Cola (Queue)**.



¿Qué es una Cola?

- Es un búfer FIFO (First In, First Out): el primer dato que entra es el primero que sale.
- Puede almacenar un número finito de elementos de tamaño fijo.
- Se definen por su longitud (cuántos elementos caben) y el tamaño de cada elemento (en bytes).
- Es vital el uso de este instrumento en RTOS y unas de sus principales ventajas



Ventajas

El uso de colas permite solucionar problemas planteados en las variables globales como:

- **Comunicación Segura (Thread-Safe):** no hay problema de condiciones de Carrera
- **Desacoplar Tareas:**
 - Enviar datos sin tener en cuenta cuando serán recibidos.
 - Verificar fácilmente si hay datos para leer
- **Diseño eficiente y orientado a eventos:** Bloqueos en lectura como escritura sin ocupar recursos del procesador.
- **Simplificar Gestión de Memoria:** el RTOS se encarga de la gestión de memoria, evitamos problemas de visibilidad de variables locales, crear mas o menos variables globales, etc

Almacenamiento de Datos: "Copia por Valor"

Has dos formas en que se puede implementar el comportamiento de la cola

Implementada por FreeRTOS



- **Por Copia:** los datos enviados a la cola se copian byte a byte
- **Por Referencia:** se guardan en la cola solo los punteros

Ventajas

- El emisor y el receptor están completamente desacoplados.
- El emisor puede modificar o liberar su variable original inmediatamente después de enviarla.
- Es seguro enviar variables locales (del stack).
- Mandar un puntero como valor → **Por referencia**



¿Quién puede usar la Cola?

Cualquier tarea (o interrupción) que conozca el "*handle*" (manejador) de la cola puede acceder a ella.

- **Caso mas común:** múltiples escritores (productores) en la misma cola.
- **Menos común pero posible:** Tener múltiples lectores (consumidores).



Bloqueo y Funciones de la API - Lectura

¿Qué pasa si la cola no contiene datos ?

Caso: Una tarea intenta leer una cola vacía → se bloquea por un tiempo determinado.

Proceso:

- Estado ***blocked*** → no consume tiempo de CPU.
- Pasa a ***ready*** cuando una interrupción o tarea coloca datos en la cola, o cuando expire su tiempo de espera.
- Si tenemos varias tareas lectoras, se desbloquea:
 - La de mayor prioridad
 - La que estuvo mas tiempo esperando (para igual prioridad)



Bloqueo y Funciones de la API - Escritura

¿Qué pasa si la cola no tiene mas lugar ?

Caso: Una tarea intenta escribir en una cola llena → se bloquea.

Proceso:

- Estado ***blocked*** → no consume tiempo de CPU.
- Pasa a ***ready*** cuando otra tarea libera espacio en la cola o expire su tiempo de espera.
- Si tenemos varias tareas escritoras, se desbloquea
 - La de mayor prioridad
 - La que estuvo mas tiempo esperando (para igual prioridad)



Creación de Colas: xQueueCreate()

```
QueueHandle_t xQueueCreate(UBaseType_t uxQueueLength,  
                           UBaseType_t uxItemSize);
```

- Parámetros:
 - **uxQueueLength:** número máximo de elementos que la cola puede contener.
 - **uxItemSize:** El tamaño en bytes de cada elemento.
- Valor devuelto
 - handle o null si no pudo crearse
- Ejemplo:

```
xQueueCreate(10, sizeof(uint32_t)); // 10 enteros
```



Envío a Colas: xQueueSendToBack()

```
 BaseType_t xQueueSendToBack( QueueHandle_t xQueue,  
                             const void *pvItemToQueue, TickType_t xTicksToWait );
```

- Envía un dato al final de la cola idem a xQueueSend()
- Parámetros:
 - **xQueue**: El identificador de la cola
 - **pvItemToQueue**: Puntero del dato a enviar (copiar).
 - **xTicksToWait**: Tiempo de espera si la cola está llena.
 - 0 para no esperar,
 - portMAX_DELAY para esperar indefinidamente.
- Retorno
 - pdPASS se enviaron los datos correctamente
 - errQUEUE_FULL no se pudo enviar los datos



Envío a Colas: xQueueSendToFront()

```
BaseType_t xQueueSendToFront( QueueHandle_t xQueue,  
                             const void *pvItemToQueue, TickType_t xTicksToWait);
```

- Similar a xQueueSendToBack() pero escribe los datos al inicio de la cola



Recepción de Colas: xQueueReceive()

```
BaseType_t xQueueReceive(QueueHandle_t xQueue, void *pvBuffer,  
                        TickType_t xTicksToWait);
```

Recibir (leer) un elemento de una cola

- Parámetros Clave:
 - **xQueue:** El identificador de la cola
 - **pvBuffer:** Un puntero a una variable donde se copiará el dato recibido.
 - **xTicksToWait:** El tiempo máximo que la tarea esperará (bloqueada) si la cola está vacía.
- Retorno
 - **pdPASS** se recibió el dato correctamente.
 - **errQUEUE_EMPTY** no se recibió ningún dato.



Nro Elementos: uxQueueMessagesWaiting()

```
UBaseType_t uxQueueMessagesWaiting(QueueHandle_t xQueue);
```

Consulta la cantidad de elementos sin leer que posee la cola

- Parámetros Clave:
 - **xQueue:** El identificador de la cola
- Retorno
 - Número de elementos que hay actualmente en la cola



Ejemplo 10 – Varias tareas escritoras

- Tenemos una serie de tareas que envían datos a una cola
- Una tarea que lee datos de la cola y lo procesa
- Además:
 - Las tareas de envío no se especifica tiempo de bloqueo, la tarea lectora si
 - Las tareas que envían datos tiene menor prioridad que la tarea lectora.
Esto indica que al momento de escribirse un dato, la tarea lectora se desbloquea y lo procesa (solo tendremos 1 datos en la cola)

Implementación:

- Dos tareas que escriben constantemente 100 y 200 respectivamente y una tarea que recibe los datos esperando 100 ms para que estos estén disponibles (nunca debería expirar este tiempo).

Ejemplo 10 - Escenario

- Tarea de escritura

```
static void vSenderTask( void *pvParameters ) {
    int32_t lValueToSend;
    BaseType_t xStatus;
    /* los valores a escribir se pasan por parámetro */
    lValueToSend = ( int32_t ) pvParameters;
    for( ;; ) {
        /* Enviar el valor a la cola */
        xStatus = xQueueSendToBack( xQueue, &lValueToSend, 0 );
        if( xStatus != pdPASS ) {
            vPrintString( "Could not send to the queue.\r\n" );
        }
    }
}
```

Ejemplo 10 - Escenario

- Tarea de lectura

```
static void vReceiverTask( void *pvParameters ) {
    int32_t lReceivedValue;
    BaseType_t xStatus;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100 );
    for( ;; ) {
        /* en este punto debería estar vacío */
        if( uxQueueMessagesWaiting( xQueue ) != 0 ) {
            vPrintString( "Queue should have been empty!\r\n" );
        }
        xStatus=xQueueReceive(xQueue,&lReceivedValue,xTicksToWait);
        if( xStatus == pdPASS ) {
            vPrintStringAndNumber( "Received = ", lReceivedValue );
        } else {
            /*Al estar enviando datos las tareas de escritura, nunca
            debería expirar los 100 ms sin datos que leer*/
            vPrintString("Could not receive from the queue.\r\n" );
        }
    }
}
```

Ejemplo 10 - Escenario

- main() donde se crean la cola y las tres tareas

```
QueueHandle_t xQueue; /* variable de la cola */
int main( void ) {
    /* se declara la cola , hasta 5 valores de tipo int32_t. */
    xQueue = xQueueCreate( 5, sizeof( int32_t ) );
    if( xQueue != NULL ) {
        /* se crean las dos tareas de envío una para enviar 100 y la
           otra 200 ambas con prioridad 1 */
        xTaskCreate(vSenderTask, "Sender1", 1000, (void*)100, 1, NULL);
        xTaskCreate(vSenderTask, "Sender2", 1000, (void*)200, 1, NULL);

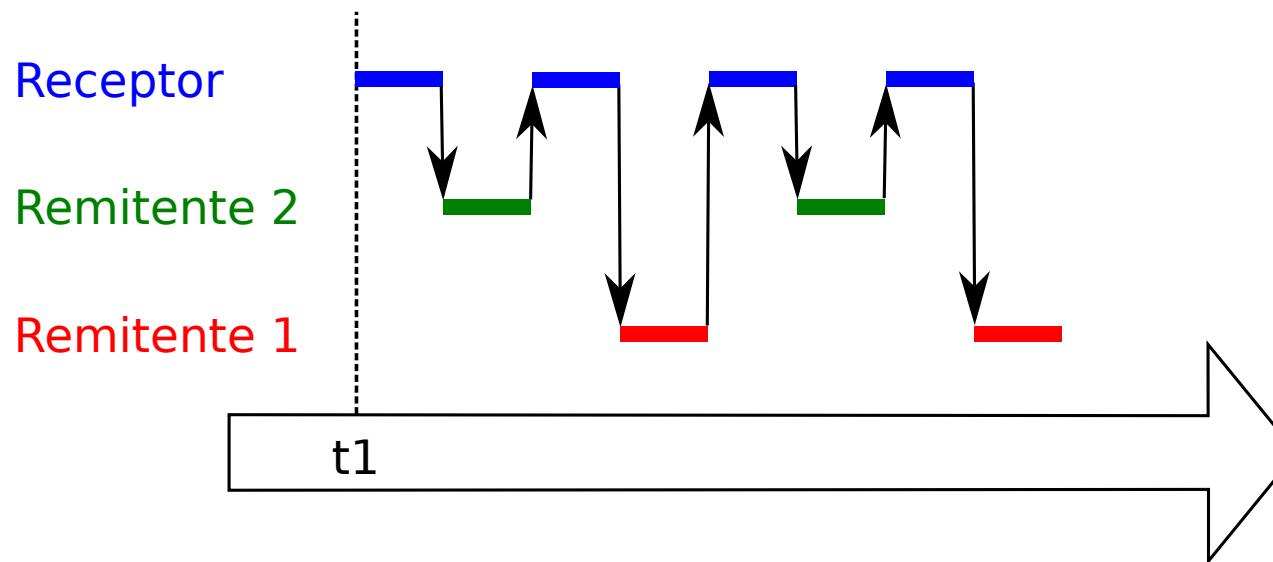
        /* se crea la tarea de lectura con prioridad 2 */
        xTaskCreate(vReceiverTask, "Receiver", 1000, NULL, 2, NULL );

        vTaskStartScheduler();
    } else {
        /* no se pudo crear la cola */
    }
    for( ;; );
}
```

Ejemplo 10 - Escenario

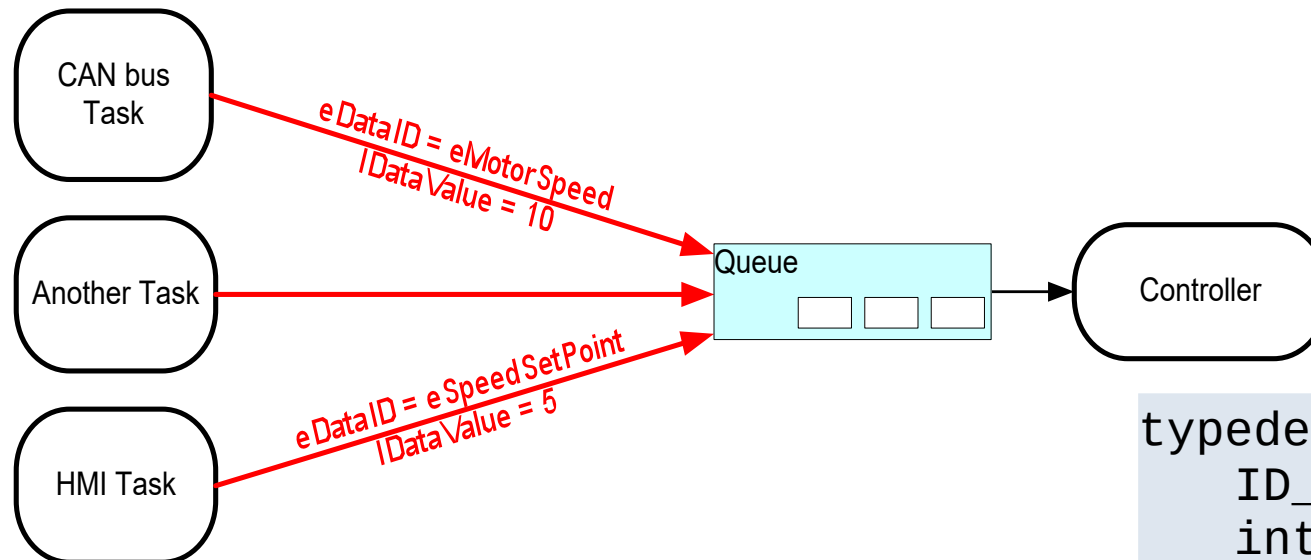
- Receptor se bloquea esperando datos
- El scheduler desaloja a Receptor y ejecuta a Remit 2.
- Remit 2 se ejecuta y envía un dato a la cola.
- Al enviar, el Receptor pasa a estado Ready.
- El scheduler desaloja a Remit 2 y ejecuta a Receptor.
- Receptor lee el dato y vuelve a llamar a xQueueReceive.
- El scheduler devuelve el control a las tareas de baja prioridad.

```
C:\WINDOWS\system32\cmd.exe - rtosdemo
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
```



Múltiples Fuentes, Una Cola

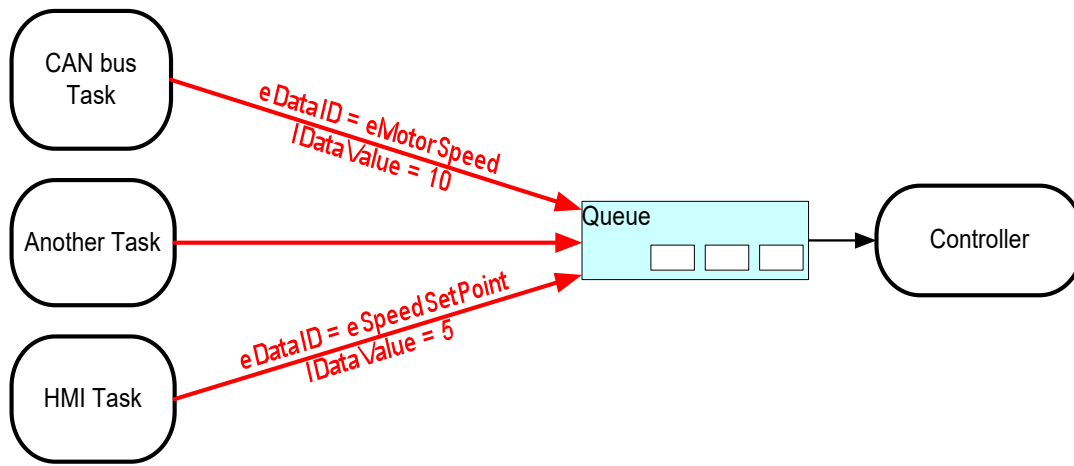
- Es común como en el ejemplo anterior, que una tarea reciba datos de mas de una fuente
- Es común también, que la tarea lectora deba saber de donde provienen esos datos o de que tipo son.
- Solución:
 - Utilizar una estructura como contenedor de datos, y guardar en uno de sus campos el origen del dato.



```
typedef struct {  
    ID_t eDataID;  
    int32_t lDataValue;  
} Data_t;
```

Múltiples Fuentes, Una Cola

- Se crea una cola de tipo `Data_t`, la cual es una estructura que contiene tanto el valor a transmitir como uno de tipo enum para el significado del dato
- Se crea una tarea para realizar el control central, la cual reacciona a las entradas y cambios en el estado del sistema.
- Una tarea “bus CAN”, cuando el bus CAN recibe un dato, lo decodifica y lo envía al controlador (velocidad del motor).
- Una tarea “interfaz hombre-maquina” (HMI), permite al operador ingresar comandos para enviarlo al controlador y consultar valores.



```
typedef struct {  
    ID_t eDataID;  
    int32_t lDataValue;  
} Data_t;
```



Ejemplo 11: Bloqueo al enviar una cola y envío de estructura

- Similar al ejemplo 10, pero ahora se invierte las prioridades para ver cómo se comporta una cola cuando los productores son más rápidos que el consumidor.
- Adicionalmente, enviaremos una estructura en lugar de un dato entero
- Configuración:
 - 2 Tareas "Sender" (alta prioridad 2).
 - 1 Tarea "Receiver" (baja prioridad 1).
- ¿ Que pasará ? : Los "Senders" llenarán la cola rápidamente y luego se bloquearán esperando a que el "Receiver" libere espacio. La cola debería estar casi siempre llena.

Ejemplo 11: Bloqueo al enviar una cola y envío de estructura

- Definimos primero las estructuras

```
/* se define el enum para identificar la fuente de los datos */
typedef enum { eSender1, eSender2 } DataSource_t;
/* Estructura que se pasa a la cola */
typedef struct {
    uint8_t ucValue;
    DataSource_t eDataSource;
} Data_t;
/* declaramos dos variables de tipo Data_t para pasar a la cola */
static const Data_t xStructsToSend[ 2 ] = {
    { 100, eSender1 }, /* Usado por Sender1. */
    { 200, eSender2 } /* Usado por Sender2. */
};
```

Ejemplo 11: Bloqueo al enviar una cola y envío de estructura

- La tarea de envío, se encarga de enviar datos, tiene un tiempo de bloqueo de 100ms, es decir, espera ese tiempo para que se libere espacio si encuentra la cola llena.
- Tiene alta prioridad
- El dato a enviar viene directamente de pvParameters

```
static void vSenderTask( void *pvParameters ) {
    BaseType_t xStatus;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100 );
    for( ;; ) {
        /* envía el dato o se queda esperando si cola esta llena*/
        xStatus=xQueueSendToBack(xQueue,pvParameters,xTicksToWait);
        if( xStatus != pdPASS ) {
            vPrintString( "Could not send to the queue.\r\n" );
        }
    }
}
```

Ejemplo 11: Bloqueo al enviar una cola y envío de estructura

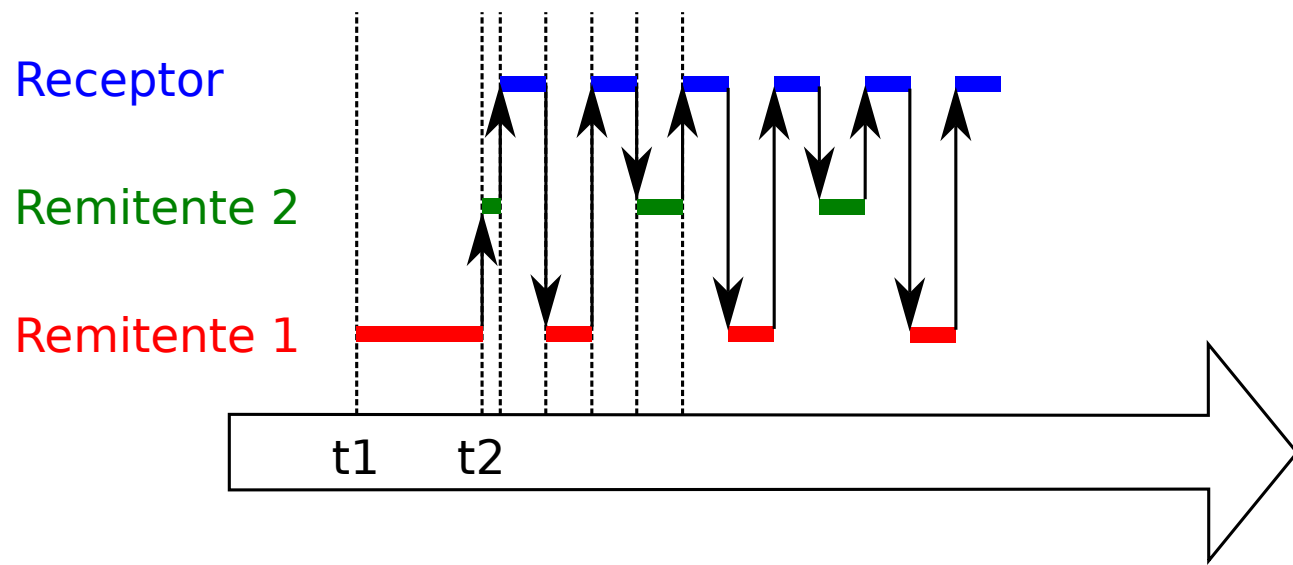
- La tarea de recepción tiene prioridad baja, es decir se ejecuta cuando las tareas de envío están bloqueadas por no tener espacio en la cola .

```
static void vReceiverTask( void *pvParameters ) {
Data_t xRStruc;
BaseType_t xStatus;
    for( ;; ) {
        if( uxQueueMessagesWaiting( xQueue ) != 3 )
            vPrintString( "Queue should have been full!\r\n" );
        xStatus = xQueueReceive( xQueue, &xRStruc, 0 );
        if( xStatus == pdPASS ) {
            if( xRStruc.eDataSource == eSender1 ) {
                vPrintStringAndNumber("From Sender 1 =",xRStruc.ucValue);
            } else {
                vPrintStringAndNumber("From Sender 2 =",xRStruc.ucValue);
            }
        } else {
            vPrintString( "Could not receive from the queue.\r\n" );
        }
    }
}
```

Ejemplo 11 - Patrón de Ejecución

- El Remitente 1 llenan la cola y se bloquean.
- El Remitente 2 es el siguiente, entra y se bloquea por lo mismo
- No hay tareas de alta prioridad en Ready, el receptor se ejecuta.
- Receptor lee UN dato, liberando un espacio en la cola.
- Esto desbloquea al Remitente que mas tiempo esperó (Remit. 1)
- El Remitente 1 entra en ejecución y vuelve a llenar la cola.

```
C:\Windows\system32\cmd.exe
C:\temp>Example011
From Sender 1 = 100
From Sender 1 = 100
From Sender 1 = 100
From Sender 1 = 100
From Sender 2 = 200
From Sender 1 = 100
From Sender 2 = 200
From Sender 1 = 100
From Sender 2 = 200
From Sender 1 = 100
From Sender 2 = 200
From Sender 1 = 100
From Sender 2 = 200
From Sender 1 = 100
```





Manejo de Datos Grandes o Variables

- Copiar estructuras grandes en una cola es ineficiente (consume tiempo y RAM).
- La solución es enviar solo un puntero al dato.
- La cola se crea con el tamaño de un puntero:
`xQueueCreate(10, sizeof(char *));`

En este caso se crea una cola de 10 elementos máximos, los elementos son simples punteros



Reglas al Enviar Punteros

- Al poner punteros en la cola, se debe tener cuidado en lo siguiente:
- **El propietario de la RAM a la que se apunta está claramente definido:** No se puede permitir que dos tareas accedan de manera simultanea al contenido de la memoria, se debe lograr que:
 - El que envía, accede a la memoria hasta que el puntero es puesto en la cola.
 - El receptor, accede a la memoria recién cuando el puntero es recibido.
- **La RAM a la que se apunta sigue siendo válida:** nunca usar puntero de variables locales, si la memoria es dinámica
 - Solo puede ser liberada por 1 tarea
 - No se debe intentar acceder una vez liberada.

Ejemplo de envío de punteros

- La tarea se encarga de enviar punteros de cadenas.

```
void vStringSendingTask( void *pvParameters ) {
    char *pcStringToSend;
    const size_t xMaxStringLength = 50;
    BaseType_t xSNumber = 0;
    for( ;; ) {
        /* función no implementada que genera un buffer */
        pcStringToSend = ( char * ) prvGetBuffer( xMaxStringLength );

        snprintf(pcStringToSend,xMaxStringLength,"String number %d\r\n",xSNumber);
        xSNumber++; // incrementa una variable que cuenta los string creados
        /* enviar el puntero a la cola */
        xQueueSend( xPointerQueue,&pcStringToSend,portMAX_DELAY );
    }
}
```

Ejemplo de envío de punteros

- La tarea que recibe el puntero se encarga luego de liberar el espacio

```
void vStringReceivingTask( void *pvParameters ) {  
    char *pcReceivedString;  
    for( ;; ) {  
        /* se toma el puntero de la cadena desde la cola */  
        xQueueReceive( xPointerQueue,&pcReceivedString,portMAX_DELAY );  
        /* se imprime el mensaje */  
        vPrintString( pcReceivedString );  
        /* se libera el buffer (función no implementada) */  
        prvReleaseBuffer( pcReceivedString );  
    }  
}
```



Patrón de Envío de Punteros

- **Tarea Emisora:**

- Asigna memoria: `Dato *pDato = pvPortMalloc(...)`.
- Rellena el dato: `pDato->valor = 123;`.
- Envía el puntero: `xQueueSend(cola, &pDato, ...);`.

- **Tarea Receptora:**

- Recibe el puntero: `xQueueReceive(cola, &pDatoRecibido, ...);`.
- Usa el dato: `procesar(pDatoRecibido->valor);`.
- Libera la memoria: `vPortFree(pDatoRecibido);`.



¿ Preguntas ?